

Práctica U8

Ejercicio Integrador 1: "Sistema de Control de Peajes Inteligente"

Objetivo: Combinar una jerarquía de clases abstractas con interfaces, forzar el uso estricto de constructores seguros e implementar la captura de errores de validación mediante excepciones personalizadas.

Contexto:

Se necesita modelar el sistema de cobro de una cabina de peaje. Por la cabina pasan distintos tipos de vehículos, y cada uno calcula su tarifa de forma diferente.

Guía Paso a Paso para el Alumno:

Paso 1: El Contrato (Interfaces)

- Crea una interfaz llamada `Tarifable`.
- Define un único método abstracto: `double calcularPeaje();`

Paso 2: La Excepción de Dominio

- Crea una clase `VehiculoInvalidoException` que herede de `Exception` (Excepción Checked).
- Debe contener un constructor que reciba un `String mensaje` y llame a `super(mensaje)`.

Paso 3: La Clase Base y el Encapsulamiento Estricto

- Crea una clase abstracta `Vehiculo`.
- Atributos estrictamente `private`: `String patente` y `String marca`.
- Regla de Cátedra: En el constructor, usando los setters, valida que ni la patente ni la marca sean `null` o estén vacías (`""`). Si lo están, debes usar `throw new VehiculoInvalidoException(...)`.
- Provee los getters correspondientes.

Paso 4: Las Implementaciones Concretas

- Crea la clase `Auto` que herede de `Vehiculo` e implemente `Tarifable`.
- En su constructor, la primera línea debe ser obligatoriamente `super(...)`.
- El peaje para un auto es fijo: \$500. Implementa el método con `@Override`.

- Crea la clase `Camion` que herede de `Vehiculo` e implemente `Tarifable`.
- Agrega el atributo `private int cantidadEjes`. Valida en su constructor que los ejes sean mayores a 2.
- El peaje es:
- $500 + (200 * \text{cantidad de ejes})$.

Paso 5: La Clase Gestora y el Programa Principal (Gestión de Errores)

- En tu `main`, crea un `ArrayList<Tarifable> vehiculosRegistrados`.
- Mediante un bucle y usando el estándar de ingesta limpia (`Scanner.nextLine()` obligatoriamente), pide al usuario que ingrese los datos para registrar un Camión.
- Manejo de Excepciones en Cascada:
- Atrapa `NumberFormatException` si el usuario ingresa letras en lugar de la cantidad de ejes.
- Atrapa `VehiculoInvalidoException` si la patente está vacía o los ejes son incorrectos.
- Si ocurre un error, muestra un mensaje amigable y vuelve a pedir los datos sin que el programa colapse.
- Al final, itera la colección polimórfica y muestra el total recaudado.

Ejercicio Integrador 2: "Gestión de Personal y Bonificaciones"

Objetivo: Aplicar herencia múltiple de tipos (implementar varias interfaces), dominar el operador `instanceof` para downcasting seguro, y manejar excepciones usando `Multicatch`.

Contexto:

Una empresa de software requiere un sistema para liquidar sueldos y evaluar a sus empleados. Algunos empleados solo cobran sueldo, otros reciben bonos, y otros además deben ser auditados.

Guía Paso a Paso para el Alumno:

Paso 1: Múltiples Contratos

- Crea la interfaz `Bonificable` con el método `double calcularBono();`.
- Crea la interfaz `Auditable` con el método `void realizarAuditoria();`.

Paso 2: La Excepción de Negocio

- Crea una clase `SalarioInvalidoException` que herede de `RuntimeException` (Excepción Unchecked).
- Esta excepción se usará para evitar sueldos negativos o inferiores al salario mínimo.

Paso 3: Jerarquía Base

- Crea la clase abstracta `Empleado` con atributos `private String nombre` y `private double sueldoBase`.
- Regla de Cátedra: En el constructor (setter), si el `sueldoBase` es menor a \$300,000, lanza la `SalarioInvalidoException`. Implementa getters.

Paso 4: Especialización y Roles Múltiples

- Crea la clase `Programador` que extienda de `Empleado` e implemente `Bonificable`.
- Bono: 15% de su sueldo base (usa el getter heredado).
- Crea la clase `GerenteIT` que extienda de `Empleado` e implemente *ambas* interfaces (`Bonificable`, `Auditable`).
- Bono: 30% de su sueldo base.

- Auditoría: Imprime un mensaje por consola indicando que se revisaron sus métricas.

Paso 5: Ingesta de Datos y Polimorfismo Seguro

- En tu `main`, crea una colección `List<Empleado> nomina`.
- Pide al usuario que ingrese datos para 1 Programador y 1 GerenteIT usando exclusivamente `Scanner.nextLine()` y `Double.parseDouble()`.
- Uso de Multicatch: En la lectura de datos, utiliza una sola estructura de rescate agrupando errores lógicos y numéricos: `catch (SalarioInvalidoException | NumberFormatException e)`.
- Iteración Polimórfica:
- Itera la `nomina`.
- Muestra el sueldo final de cada empleado (sueldo base + bono). *Pista: Necesitarás verificar si el empleado instanceof Bonificable y hacer un downcasting seguro para poder llamar a calcularBono().*
- Si el empleado es `Auditable`, ejecuta su método `realizarAuditoria()`.

Teoría Fundamental: Lectura Eficiente y Gestión de Memoria

El Problema de los Recursos Abiertos:

Cuando nuestro programa en Java se conecta al disco duro para leer un archivo, establece un "puente" de comunicación. Si ocurre un error a mitad de la lectura (por ejemplo, el archivo se corrompe o se desconecta el disco), el programa lanza una `IOException`. En el pasado, si no cerrábamos ese puente manualmente en el bloque `finally`, el archivo quedaba bloqueado en la memoria (Memory Leak).

****La Solución Moderna: `try-with-resources`****

A partir de Java 7, se introdujo el `try-with-resources`. Cualquier objeto que implemente la interfaz `AutoCloseable` (como los lectores de archivos) puede ser instanciado dentro de los paréntesis del `try`. La Máquina Virtual de Java (JVM) asume el contrato absoluto de cerrar ese recurso automáticamente al terminar el bloque o al ocurrir un error, haciendo obsoleto e innecesario el uso del bloque `finally` para esta tarea.

****El Patrón de Lectura: `FileReader + BufferedReader`****

Para leer un archivo de texto plano como un `.csv` (Valores Separados por Comas), no leemos byte por byte (es muy lento). Usamos un `FileReader` para conectarnos al archivo, y lo envolvemos en un `BufferedReader`. Este último actúa como una sala de espera (búfer) que trae bloques grandes de texto a la RAM, permitiéndonos leer el archivo línea por línea de forma extremadamente rápida con su método `.readLine()`.

Archivo de Prueba: `productos.csv`

Entrégale este bloque de texto a tus alumnos. Deben crear un archivo llamado `productos.csv` en la raíz de su proyecto (fuera de la carpeta `src`).

```
101,Laptop Gamer,1500000.0,Fisico
```

```
102,Suscripcion Premium,5000.0,Digital
```

```
103,Mouse Inalambrico,25000.0,Fisico
```

```
104,Licencia Antivirus,8000.0,Digital
```

```
105,Teclado Mecanico,45000.0,Fisico
```



Ejercicio Integrador 3: "Importación Masiva de Catálogo"

Objetivo: Crear un motor de importación que lea un archivo CSV mediante `try-with-resources`, parsee cada línea inyectando los datos en objetos que implementen una interfaz, y controle estrictamente los errores de formato o archivos inexistentes.

Guía Paso a Paso para el Alumno:

Paso 1: El Contrato de Importación (Interfaz)

- Crea una interfaz llamada `Importable`.
- Define el método abstracto: `void cargarDesdeFila(String filaCsv) throws Exception;`. Este método obligará a las clases a saber cómo armarse a sí mismas a partir de un texto separado por comas.

Paso 2: La Clase de Dominio

- Crea la clase `Producto` que implemente la interfaz `Importable`.
- Atributos estrictamente `private`: `int id`, `String nombre`, `double precio`, `String tipo`.
- Implementa (con `@Override`) el método `cargarDesdeFila(String filaCsv)`.
- *Pista de cátedra:* Usa el método `filaCsv.split(",")` para obtener un arreglo de `String` con las 4 columnas.
- Asigna los valores a los atributos transformando los datos numéricos con `Integer.parseInt()` y `Double.parseDouble()`.
- Validación: Si el precio parseado es menor a 0, lanza manualmente una `IllegalArgumentException("El precio no puede ser negativo")`.

Paso 3: El Motor de Lectura Segura (Main)

- En tu clase principal, crea un método estático `importarCatálogo(String rutaArchivo)`.
- Crea un `ArrayList<Producto> catalogo = new ArrayList<>();`.
- Aplica `try-with-resources`: Instancia el `BufferedReader` dentro de los paréntesis del `try`.

```
try (BufferedReader br = new BufferedReader(new
FileReader(rutaArchivo))) {
```

```
// Lógica de lectura
```

```
}
```

- Lectura Cíclica: Crea una variable `String linea;` y un bucle `while ((linea = br.readLine()) != null).`
- Dentro del bucle, por cada línea:
- Instancia un `new Producto()`.
- Llama al método `.cargarDesdeFila(linea).`
- Agrega el producto al `ArrayList`.

Paso 4: Manejo de Excepciones en Cascada (Multicatch)

- El bloque `try-with-resources` debe estar escoltado por múltiples `catch`.
- Primer Catch (`FileNotFoundException`): Atrapa si el archivo `productos.csv` no existe en la ruta indicada. Imprime: *"Error Crítico: No se encontró el archivo de base de datos."*
- Segundo Catch (`IOException`): Atrapa si hay un error físico de lectura.
- Tercer Catch (Manejo de Datos Corruptos): Usa Multicatch para atrapar `NumberFormatException` | `IllegalArgumentException`. Si alguna línea del CSV tiene letras en lugar de números o precios negativos, informa: *"Error de integridad de datos en la importación. Verifique el formato del CSV."*

Paso 5: Ejecución y Comprobación

- En tu `main`, llama a `importarCatálogo("productos.csv").`
- Itera el `ArrayList` resultante e imprime el catálogo formateado para demostrar que los objetos se crearon exitosamente en la memoria.